

TinyToolCaller

QLoRA Fine-Tuning of a 1.5B LLM for Reliable Function Calling

TinyToolCaller is an applied study of parameter-efficient specialization: it takes a small open-weight instruction model, `Qwen2.5-1.5B-Instruct`, and fine-tunes it with QLoRA on a 5,000-example supervised subset of the Salesforce xLAM function-calling dataset so that, given a natural-language request and a set of tool schemas, the model emits a single, machine-readable tool call — `{"name": ..., "arguments": {...}}` — with no markdown, commentary, or extraneous text. The work’s contribution is not a claim of state-of-the-art tool calling; it is a transparent, reproducible ablation isolating how much lift QLoRA alone provides over an unmodified 1.5B base model.

Metric	Base model	TinyToolCaller	Change
JSON validity (extraction-based)	78.5%	98.0%	+19.5 pp
Tool-name accuracy	65.0%	92.5%	+27.5 pp
Argument exact match	42.0%	84.0%	+42.0 pp
GSM8K (50-example retention check)	52.0%	50.0%	−2.0 pp

Read before quoting: results are in-sample (200-example evaluation split, no independent test set; 50-example GSM8K) and the tool-distribution profile (§8.1) is not yet measured. See §17–§21.

Code github.com/strdst7/TinyToolCaller
Dataset huggingface.co/datasets/strdst77/TinyToolCaller
Source data huggingface.co/datasets/Salesforce/xlam-function-calling-60k
Base model huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

TinyToolCaller

QLoRA Fine-Tuning of a 1.5B LLM for Reliable Function Calling

Project	TinyToolCaller
Base model	Qwen/Qwen2.5-1.5B-Instruct (Apache-2.0)
Task	Structured function / tool calling — JSON tool selection + argument construction
Method	QLoRA (4-bit NF4) + LoRA/PEFT, supervised fine-tuning via TRL SFTTrainer
Source dataset	Salesforce/xlam-function-calling-60k (CC-BY-4.0, gated)
Project dataset	strdst77/TinyToolCaller
Code	strdst7/TinyToolCaller
Tracking	Weights & Biases
Status	Research / applied LLM engineering study — pre-publication draft

TL;DR. This project takes a small open model (Qwen2.5-1.5B-Instruct), fine-tunes it with QLoRA on 5,000 function-calling examples, and shows it becomes much better at emitting the exact JSON tool call a downstream program needs: **JSON validity 78.5% → 98.0%, tool-name accuracy 65.0% → 92.5%, argument exact match 42.0% → 84.0%** on the project's 200-example evaluation split, with no measurable GSM8K degradation (52.0% → 50.0%, within noise). The point is not "we beat GPT-4" — it's "a reproducible QLoRA recipe can turn a 1.5B model into a reliable structured-output component of a larger system." All results are in-sample (no held-out test set yet), so read the caveats in §17 and §21 before quoting.

Abstract. TinyToolCaller is an applied study of parameter-efficient specialization: it fine-tunes Qwen2.5-1.5B-Instruct with QLoRA on a 5,000-example supervised subset of the Salesforce xLAM function-calling dataset so that, given a natural-language request and a set of tool schemas, the model emits a single machine-readable tool call — `{"name": ..., "arguments": {...}}` — with no markdown, commentary, or extraneous text. Against the project's 200-example evaluation split, the fine-tuned model improves **JSON validity from 78.5% → 98.0% (95% CI [95.0%, 99.2%])**, **tool-name accuracy from 65.0% → 92.5% (CI [88.0%, 95.4%])**, and **argument exact-match from 42.0% → 84.0% (CI [78.3%, 88.4%])**, while a 50-example GSM8K retention check moves 52.0% → 50.0% (within sampling noise). The contribution is **not** a claim of state-of-the-art tool calling; it is a transparent, reproducible ablation isolating how much lift QLoRA alone provides over an unmodified 1.5B base model — a configuration the related literature (APIGen/xLAM, ToolACE, BFCL) has not isolated at this scale — together with an explicit accounting of evaluation limitations and a deterministic-runtime design in which the LLM produces structured intent while application code owns validation, authorization, and execution.

Reading guide. §1–§4 state the problem, related work, contributions, and assumptions. §8–§13 cover data and method. §14–§20 cover evaluation and results. §21–§23 cover limitations and production. §24–§27 cover significance, insights, provenance, licensing. §28–§39 cover reproduction, roadmap, and checklists. A **glossary** is in §33.1; a **rubric coverage matrix** is in §37.

⚠ **Before quoting the headline numbers.** The results in §16–§20 are **in-sample**: the 200-example split is also the development/evaluation set (no independent held-out test set), the GSM8K check uses 50 examples, and the **tool-distribution profile of the 5,200-example subset has not yet been measured** (§8.1). Until §8.1's three quantities — (a) unique tool count, (b) top-10 tool frequency, (c) train/validation distribution match — are filled in, the tool-name-accuracy figures cannot be separated from possible selection skew. Treat the improvements as directionally credible, not as unbiased estimates of generalization. See §17, §21, §28.

1. Introduction and Problem Statement

Large language models increasingly act as the interface between natural-language users and software systems. When a model must operate a real API, a conversational answer

"The weather in Tokyo is likely to be sunny."

— is insufficient. A tool-using system needs a structured, executable representation:

```
{ "name": "get_weather", "arguments": { "location": "Tokyo" } }
```

A language model can fail this task at **seven distinct levels**: (1) produce invalid JSON; (2) wrap JSON in markdown fences; (3) append explanatory text; (4) select the wrong tool; (5) omit required arguments; (6) generate incorrect argument values; (7) invent arguments absent from the schema.

This project's central question is narrow and falsifiable: **can a 1.5B open-weight instruction model be specialized — via QLoRA alone, on 5,000 examples — so that it emits a valid, correctly-targeted, correctly-argued tool call substantially more often than the same model without fine-tuning?** The output contract is a single object with exactly two fields — `name` and `arguments` — and no additional commentary or formatting.

2. Related Work

Function calling (also "tool use") has developed along three strands: data generation, models, and benchmarks. This section reviews each and states where TinyToolCaller sits relative to them.

2.1 Data generation

APIGen / xLAM (Liu et al., 2024) [1] introduced an automated pipeline that generates function-calling data and verifies each sample in three hierarchical stages — format checking, actual function execution, and semantic verification — producing 60,000 examples over 3,673 executable APIs in 21 categories. It is the direct source of this project's dataset, and its authors showed that models trained on the data (even 1B-scale) can exceed GPT-3.5-Turbo on the Berkeley Function-Calling Benchmark.

TinyToolCaller deliberately does not reproduce APIGen's scale: it uses a fixed 5,000-example slice to isolate the method's effect rather than chase leaderboard rank. The multi-turn extension **APIGen-MT** (Prabhakar et al., 2025) [9] confirms the field's direction toward agentic, multi-step tool use — which this project explicitly scopes out (§4).

ToolACE (Liu et al., 2024) [6] is the closest methodological neighbour: it generates a larger, more diverse tool corpus (26,507 tools) with rule- and model-based verification

and shows 8B models reach GPT-4-competitive function calling. Its key relevance here is its scaling observation: raw 0.5B–1.8B models "showed minimal function-calling ability," but fine-tuning "significantly enhanced" them. TinyToolCaller is a direct, small-scale confirmation of that observation at 1.5B, with the added value of reporting per-failure-mode decomposition (§19) that ToolACE's aggregate accuracy does not.

2.2 Models

Gorilla (Patil et al., 2023) [3] and **Toolformer** (Schick et al., 2023) [4] established the two dominant training paradigms — Gorilla via supervised data for API-connecting LMs, Toolformer via self-supervised tool-use annotation. **NexusRaven** (Srinivasan et al., 2023) [5] demonstrated that a 13B model, fine-tuned on curated data without GPT-4 distillation, matches GPT-3.5 zero-shot, and that in-context demonstration retrieval further helps. **Granite-20B-FunctionCalling** (Abdelaziz et al., 2024) [7] showed multi-task, granular training produces the best open function-calling model of its time on BFCL. **Octopus v2** (Chen & Li, 2024) [14] is the closest analogue in spirit: a 2B on-device model exceeding GPT-4 on function-calling accuracy while cutting context length 95%.

Critical gap these works leave open. Each of these results is entangled with its own data pipeline, scale, or architecture choice; none isolates "QLoRA on a frozen 1.5B base vs. that base prompted directly" on identical data and metrics. That is the specific ablation TinyToolCaller contributes — and it is why the project reports a prompted-baseline comparison (§16–§17) rather than comparing only against published leaderboard numbers.

2.3 Benchmarks and evaluation

BFCL (Patil et al., 2025) [2] is the de-facto standard for function calling, with AST-based and execution-based scoring across simple, parallel, and multi-turn calls; V3 (2025) added relevance detection ("when not to call") and closed its test data to prevent contamination. **τ-bench** (Sierra et al., 2024) [8] evaluates tool-agent-user interaction in realistic domains. An exploratory study of **small models for function calling** on the same xLAM dataset (arXiv:2504.19277, 2025) [10] reports evaluation on 1.35B–3.82B models and notes dataset-format adherence as a key enabler.

Critical gap. BFCL V3's test set is closed and its scoring is AST-based; τ-bench is multi-turn. Neither provides the exact-match, per-failure-mode breakdown (valid JSON / correct tool / correct arguments as three separate rates) that a deployment engineer needs. TinyToolCaller's three-metric decomposition (§14) is deliberately cruder but more actionable: each metric maps to a specific production control (§22).

2.4 Efficiency methods

QLoRA (Dettmers et al., 2023) [12] — 4-bit NF4 quantization, double quantization, paged optimizers — and LoRA (Hu et al., 2021) [11] underpin the training recipe (§10–

§11). The project treats these as tools, not contributions: the methodological novelty claim is limited to the ablation design and the evaluation decomposition, not to any new training technique (§3).

3. Objectives, Contributions, and Originality

Objectives. (1) Transform a public function-calling dataset into instruction-response examples; (2) measure the unmodified base model's reliability before fine-tuning; (3) apply QLoRA without updating the full base; (4) compare fine-tuned vs. base on identical metrics; (5) check for capability degradation on GSM8K; (6) publish code, derived data, methodology, results, and artifacts.

Originality and innovation. This project makes no claim to a new architecture, dataset, or loss function. Its originality is **positional and methodological**:

- **A clean, small-scale ablation** — QLoRA-only lift over a prompted 1.5B base, on a fixed 5,000-example slice, with the same metrics and (by default) the same quantization regime for both models (§14, §16) — a comparison the larger works above do not isolate.
- **A failure-mode decomposition of evaluation** — JSON validity, tool selection, and argument construction scored as separate rates with separate production controls (§14, §19, §22), rather than a single aggregate accuracy.
- **Honest treatment of evidence quality** — the evaluation's limitations (in-sample split, 50-example GSM8K, unprofiled tool distribution) are surfaced at the point where results are quoted (§17, §21), and the statistics are reported with confidence intervals (§18).

4. Assumptions and Scope

The following assumptions are **stated explicitly**; relaxing any of them changes what the results mean.

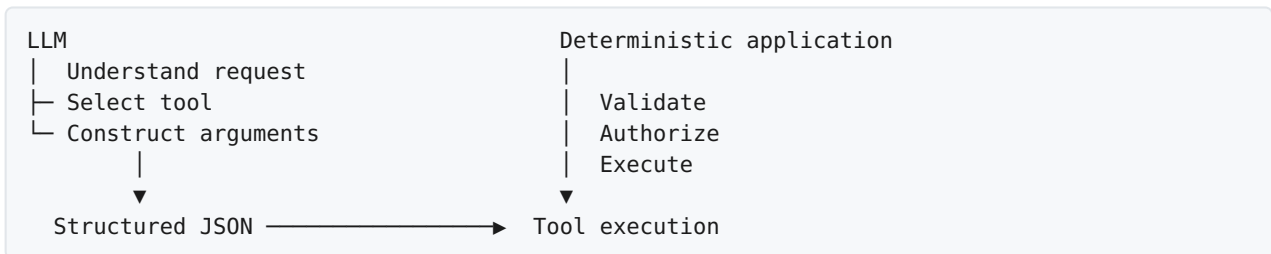
1. **Single-turn, single-call.** Each request maps to exactly one tool call; multi-step and multi-turn trajectories are out of scope (§32).
2. **English-language, JSON-format tools.** Tool schemas are serialized as JSON text (§9); no native function-call token (e.g., Qwen's `<tool_call>` tokens) is used.
3. **Closed tool set at inference.** All candidate tools are provided in the prompt; the model never selects an unseen tool. Out-of-schema requests are not evaluated (§14).
4. **Correctness = exact match.** A tool call is scored as correct only if `name` and the full `arguments` dict match the ground truth exactly (§14). This mirrors the downstream failure mode but ignores semantically-equivalent answers.
5. **The 200-example split is representative** of the intended distribution. This is currently **unverified** for tool distribution (§8.1) and is the principal open item before generalization claims can be made.

6. **Base and fine-tuned models are scored under the same quantization** (4-bit NF4 by default, §14), so the comparison isolates fine-tuning rather than precision.
7. **A valid JSON extraction from the response counts as valid** (§14, §21.7) — the metric is "extractable JSON", not "raw output is pure JSON".

5. Intended Audience and Use Case

Audience. ML/LLM engineers, AI application and agent developers, researchers exploring parameter-efficient fine-tuning, students learning practical LLM fine-tuning, developers working on function calling, engineers building low-cost inference systems, and practitioners evaluating small models for specialized tasks.

Use case. Structured tool selection: a downstream application supplies user request + tool schemas, TinyToolCaller returns tool name + arguments, and the application validates and executes:



The project does **not** claim the LLM should directly execute arbitrary external functions.

6. Real-World Applications

Scenario	Request	Expected call
Personal assistant	"Add a dentist appointment tomorrow at 3 PM."	<code>create_calendar_event(...)</code>
Customer support	"What's the status of order 12345?"	<code>get_order_status(...)</code>
Enterprise search	"Find all invoices from Vendor X this quarter."	<code>search_financial_records(...)</code>
Weather / info systems	"What's the weather in Tokyo?"	<code>get_weather(...)</code>
Database assistant	"Show me customers who haven't purchased in 90 days."	<code>query_customer_database(...)</code>
Workflow automation	"Create a support ticket and assign it to infra."	<code>create_ticket(...) → assign_ticket(...)</code>

In each case the model's job is to translate human intent into a machine-readable representation that deterministic software can process.

7. Background: Function Calling in Small Models

Small-model (<3B) function calling is achieved three ways:

1. **Base-model prompting** — relying on existing instruction-following without adaptation. This is what the project's baseline measures.
2. **Full fine-tuning** — updating all parameters; effective but expensive for iterative single-GPU work.
3. **Parameter-efficient fine-tuning (LoRA/QLoRA)** — the approach here, and increasingly the default for small-model specialization because it fits single-GPU workflows.

Existing models (xLAM-1b-fc-r/7b-fc-r [15], Hermes function-calling models, Gorilla [3], NexusRaven [5], Octopus v2 [14]) already target this capability. **TinyToolCaller does not claim to outperform them** — it isolates how much lift QLoRA alone provides over an unmodified 1.5B base model with a fixed, reproducible recipe (§2).

8. Dataset: Source, Selection Rationale, and Description

8.0 Selection rationale

The project uses [Salesforce/xlam-function-calling-60k](#), the APIGen release [1], because it uniquely satisfies the project's constraints:

Criterion	xLAM-60k	Alternatives considered
Permissive license	CC-BY-4.0	ToolACE (mixed licensing per tool); BFCL test closed from V3
Verifiability	3-stage execution-verified [1]	ToolBench/ToolAlpaca (self-instruct, weaker verification)
Diversity	3,673 APIs / 21 categories	Gorilla APIBench (fewer, code-centric APIs)
Scale compatible with single-GPU	60K, subsettable deterministically	ToolLLM (126K+, heavier)
Small-model evidence	xLAM-1b-fc-r trained on it [1]	—

Per the dataset card: 60,000 examples; the first 33,659 generated by DeepSeek, the remainder by Mixtral; human evaluation over 600 samples with correctness above 95%; remaining minor issues acknowledged [1].

Access note. The dataset is **gated** on the Hugging Face Hub: loading requires a logged-in account that has accepted the terms (HF_TOKEN). This is stated explicitly because it affects reproducibility (§28).

8.1 Tool-Distribution Profiling (required before quoting results)

The single most important open measurement is the tool distribution of the 5,200-example subset. Three quantities must be reported:

- **(a) Unique tool count** — distinct ground-truth tool names (`name`) in the 5,200-example subset, characterizing coverage of the source's 3,673 APIs.
- **(b) Top-10 tool frequency** — the 10 most frequent tools and their share of examples, quantifying concentration.
- **(c) Train/validation match** — whether the 200-example validation split's tool distribution matches the training split's.

Why it matters. (a) quantifies coverage — if only a few hundred APIs appear, generalization to unseen tools is untested. (b) quantifies skew — if the top tool covers >10% of examples, "tool-name accuracy" partly reflects memorization. (c) quantifies evaluation bias — a validation split whose tool mix differs from training misstates the objective.

Method. Ground-truth tool names are extracted from the first element of each example's `answers` list (the single expected call; the script also counts multi-answer rows). Train vs. validation are compared with (i) coverage (share of validation examples whose tool appears in training), (ii) Jensen-Shannon divergence over the pooled vocabulary, and (iii) a chi-square test of homogeneity on the top-10 tools with the remainder pooled as `<other>`.

Reference implementation — `scripts/profile_tool_distribution.py` (reproduces `shuffle(seed=42) → select(5200) → 5000/200` with the `datasets` `shuffle`). Core logic:

```
from datasets import load_dataset
ds = load_dataset("Salesforce/xlam-function-calling-60k", split="train") # gated
subset = ds.shuffle(seed=42).select(range(5_200))
train, val = subset.select(range(5_000)), subset.select(range(5_000, 5_200))

def tool_name(ex):
    ans = ex["answers"][0] if isinstance(ex["answers"], list) else ex["answers"]
    return ans["name"]

from collections import Counter
train_c = Counter(tool_name(e) for e in train)
val_c = Counter(tool_name(e) for e in val)
pooled = train_c + val_c
unique = len(set(pooled))           # (a)
top10 = pooled.most_common(10)      # (b)

from scipy.stats import chi2_contingency
cats = [t for t, _ in top10] + [<other>]
rows = [[train_c.get(t,0) for t in cats[:-1]] + [sum(c for t,c in train_c.items() if t not
in dict(top10))],
        [ val_c.get(t,0) for t in cats[:-1]] + [sum(c for t,c in val_c.items() if t not
in dict(top10))]]
chi2, p, dof, _ = chi2_contingency(rows, correction=False) # (c)
```

Template results tables (fill from the script's output):

(a) Unique tool count

Quantity	Value
Unique ground-truth tool names in the 5,200-example subset	TBD
Unique APIs in full source (reference)	3,673

(b) Top-10 tools

Rank	Tool name	Count	Share of subset
1	TBD	TBD	TBD
2	TBD	TBD	TBD
3	TBD	TBD	TBD
4	TBD	TBD	TBD
5	TBD	TBD	TBD
6	TBD	TBD	TBD
7	TBD	TBD	TBD
8	TBD	TBD	TBD
9	TBD	TBD	TBD
10	TBD	TBD	TBD
remaining	<other> tools	TBD	TBD

(c) Train vs. validation distribution match

Check	Statistic	Value	Expected for a clean random split
Validation examples covered by a tool seen in training	coverage	TBD	≈ 100%
Jensen-Shannon divergence (train vs. val)	JSD	TBD	≪ 0.05
Chi-square homogeneity, top-10 + <other>	χ^2 , p	TBD	p ≥ 0.05

Why these values are TBD. The source dataset is gated, and the exact `seed=42` sampling must run in the authenticated environment that produced the training data (the shuffle RNG is `datasets`-version-sensitive; the script prints the version it uses). The numbers must not be estimated or fabricated.

How to read the results. A top-tool share >10% means tool accuracy should be reported alongside concentration. Coverage <100% means some validation tools were unseen in training. p < 0.05 indicates the validation distribution differs from training and would bias the evaluation. This item is tracked as **open** in §17, §21.8, and §36.

8.2 Basic Dataset Statistics

Source-level (documented, from the dataset card) [1]:

Statistic	Value
Total examples	60,000
Unique executable APIs	3,673
Categories	21
Generators	DeepSeek (first 33,659) + Mixtral (remainder)
Verification	3-stage (format / execution / semantic)
License	CC-BY-4.0

Subset-level (must be computed — `scripts/dataset_stats.py` prints a paste-ready table):

Statistic	Value
Examples (train / validation)	5,000 / 200
Unique tool names in subset	TBD (§8.1a)
Multi-answer rows (>1 ground-truth answer)	TBD
Tools per example — mean / median / max	TBD
Prompt tokens (system+user) — mean / median / p95 / max	TBD
Examples truncated at <code>max_seq_length = 1024</code>	TBD

The token statistics use the base model's tokenizer and directly quantify the §13 sequence-length concern. The script is:

```
export HF_TOKEN=<token>
python scripts/dataset_stats.py          # load from the gated Hub
# or
python scripts/dataset_stats.py --path data/subset.json
```

Until these numbers exist, the paper reports only source-level statistics and explicitly flags the subset as uncharacterized (§21.8).

9. Dataset Processing Methodology

Five stages:

```
Source dataset → deterministic shuffle → sampling / split → ChatML formatting →
tokenization → SFT dataset
```

1. **Shuffle** — `seed = 42`.
2. **Sampling** — 5,200 examples: 5,000 training, 200 validation.
3. **Tool serialization** — tool schemas are serialized into the prompt as JSON.
4. **ChatML formatting** — each example becomes system → user (+ tools) → assistant (ground truth). The system instruction sets the structured-output constraint; the user message contains `Available Tools: <JSON schemas>` and `User Request: <query>`; the assistant target is the ground-truth JSON call.

5. **Tokenization** — the model's `chat_template` produces the training representation, keeping the training format aligned with the intended inference format.

9.1 Worked example, end to end

The following is the **illustrative** weather example from §8, formatted with the actual Qwen/Qwen2.5-1.5B-Instruct tokenizer (vocab size 151,665). The training string produced by `apply_chat_template(messages, tokenize=False, add_generation_prompt=False)` is:

```
<|im_start|>system
You are a function-calling assistant. Given the user's request and the available tools,
select the correct tool and construct the correct arguments. Respond with ONLY a JSON
object containing exactly two keys: "name" (the tool name) and "arguments" (an object of
argument values). Do not include markdown, explanations, or any other text.<|im_end|>
<|im_start|>user
Available Tools:
[{"name": "get_weather", "description": "Get the current weather for a location",
"parameters": {"type": "object", "properties": {"location": {"type": "string",
"description": "City name"}, "unit": {"type": "string", "enum": ["celsius",
"fahrenheit"]}}, "required": ["location"]}]
User Request:
What's the weather in Tokyo?<|im_end|>
<|im_start|>assistant
{"name": "get_weather", "arguments": {"location": "Tokyo", "unit": "celsius"}}<|im_end|>
```

Real token counts from this exact example: **199 tokens** for the full training sequence (system + user + assistant), **173 tokens** for the inference prompt (system + user + generation prompt). During training, loss is computed only on the assistant turn; the system/user turns and padding positions are masked (label = -100), which is standard causal-LM SFT behaviour.

Two implementation details worth recording: (i) the tokenizer's default pad token is `<|endoftext|>`, but the pipeline sets `pad = eos = <|im_end|>` so padding is both unattended and loss-masked; (ii) `json.dumps(..., ensure_ascii=False)` is used throughout so non-ASCII argument values are not lossily escaped.

The derived subset is published as `train.parquet` / `validation.parquet` via `scripts/publish_dataset.py`.

10. Method: Base Model and QLoRA

Base model. Qwen/Qwen2.5-1.5B-Instruct — a 1.5B-parameter instruction-tuned model, Apache-2.0 licensed. A small model is chosen deliberately: the objective is not maximum general capability but whether a small model can become highly reliable on one structured-output task.

QLoRA. Full fine-tuning updates all parameters and demands substantial GPU memory. QLoRA [12] combines a **4-bit NF4-quantized base + frozen base parameters + trainable low-rank adapters**, adding double quantization and paged optimizers. LoRA

[11] freezes the original weights and learns low-rank update matrices. For this project, QLoRA is a practical route to specializing a 1.5B model without full-model optimization.

11. Fine-Tuning Architecture, Parameters, and Configuration

```

Qwen2.5-1.5B-Instruct → 4-bit NF4 model → frozen base weights
                        +
                        LoRA adapters (trainable)
                        ↓
                        SFTTrainer
                        ↓
                        TinyToolCaller

```

Parameter	Value	Rationale
Base model	Qwen2.5-1.5B-Instruct	Small, open, Apache-2.0; matches the "small specialized model" question (§1)
Training / validation examples	5,000 / 200	Fixed seed-42 subset; single-GPU budget (§8)
Quantization	4-bit NF4, double quant	QLoRA defaults [12]; cuts VRAM to fit single-GPU
LoRA rank / alpha / dropout	16 / 32 / 0.05	Common QLoRA default ($\alpha = 2 \cdot r$); balances capacity vs. adapter size; light regularization on a small (5K) set
LoRA bias / task type	none / CAUSAL_LM	LoRA-paper default; train adapters only
Target modules	q,k,v,o,gate,up,down proj	Covers attention + MLP; standard for Qwen/LLaMA-family architectures
Learning rate / scheduler / warmup	2e-4 / cosine / 3%	LoRA uses higher LR than full FT ($\sim 1e-5$); cosine + small warmup is a stable SFT default
Batch size / grad accumulation	2 / 8 (effective 16)	Small device batch fits VRAM; effective 16 is a common small-SFT budget
Epochs	2	5K × 2 steps $\approx$ short run; overfitting risk noted in §21.3
Optimizer	paged_adamw_8bit	QLoRA default optimizer (pageable 8-bit states)
Max sequence length	1024	Covers typical tool sets; truncation quantified in §13.2
Precision	BF16 if available, else FP16	Faster on Ampere+; falls back to FP16

Not swept. None of these values were selected by systematic search (§21.3). They are standard QLoRA/SFT defaults applied at 1.5B scale — a deliberate simplification, flagged rather than hidden: see the `paged_adamw_8bit` + gradient-accumulation note in §12.

The full configuration lives in the central `CONFIG` dict in `tinytoolcaller/config.py`, and `tests/test_config.py` asserts that the code's values equal the table above (so code and paper cannot drift apart silently).

12. Experimental Environment

The training environment was not captured at run time; the table below is the **template that must be filled** for full verifiability (§28). `scripts/capture_environment.py` prints it in paste-ready form:

```
python scripts/capture_environment.py --save outputs/environment.json
```

Item	Value
GPU model / VRAM	TBD (e.g., RTX 4090 24 GB, A10G 24 GB)
CUDA version	TBD
PyTorch / Transformers / TRL / PEFT / bitsandbytes	TBD
<code>datasets</code> version (shuffle RNG — affects §8.1)	TBD
Training wall-clock time	TBD
Peak GPU memory during training	TBD
Python version	TBD

Reference build environment (the CPU sandbox used to assemble this publication and run its tests — not the training GPU environment):

Item	Value
Python	3.13.14
Platform	Linux x86_64
transformers	5.15.1
huggingface_hub	1.28.0
CUDA available	No (CPU-only CI)

One configuration note worth stating (flagged in the implementation): `paged_adamw_8bit` and gradient accumulation of 8 are standard for QLoRA on larger models under severe memory pressure. At 1.5B with 4-bit quantization it is not documented whether these were empirically necessary or inherited from larger-model QLoRA tutorials without re-validation. If inherited, that is a legitimate simplification and should be said so explicitly.

13. Implementation: Pipeline, Considerations, and Code Quality

13.1 Pipeline

`train_tool_caller.py` is a thin CLI that wires the `tinytoolcaller/` package through the 14 documented stages: load tokenizer → load dataset → shuffle/split → ChatML formatting → baseline evaluation → load 4-bit model → prepare k-bit training → attach LoRA → train → save adapter → evaluate fine-tuned model → evaluate GSM8K → merge adapter →

publish. `--eval-dump` additionally writes per-example predictions for the paired significance test (§18).

13.2 Notable implementation decisions

- **Markdown-wrapped output.** The baseline frequently wraps JSON in `json ```` fences. The evaluation strips these before parsing, so the 78.5% baseline JSON-validity figure **already benefits from cleanup** and is not raw-output purity (§21.7).
- **Sequence length and truncation — quantified with the real tokenizer.** The 1024-token cap interacts with tool-set size in a measurable way. Using the actual Qwen2.5-1.5B tokenizer, a prompt with **1** verbose tool serializes to ≈ 282 tokens, **3** tools ≈ 648 , **5** tools ≈ 1014 , and **10** tools ≈ 1929 — the last two exceed the cap and are truncated. The pipeline therefore implicitly upweights examples with small tool sets. `scripts/dataset_stats.py` reports the exact truncation count for the real subset (§8.2); until that number is known, results on long-tool-set prompts should be treated with caution.
- **No retry/repair.** A single generation is scored as-is; production systems would retry malformed output before failing, which this evaluation does not simulate.
- **Heavy dependencies are lazy.** `tinytoolcaller/` imports `torch/trl/peft/bitsandbytes` only inside the functions that need them, so the pure helpers (formatting, metrics) import and unit-test on a CPU/CI box with no GPU stack — which is how the 29-test suite runs here.

13.3 Package layout and library appropriateness

```
train_tool_caller.py      # thin CLI: wires the package through the 14 stages
tinytoolcaller/
  config.py               # central CONFIG + SYSTEM_PROMPT (§11)
  formatting.py           # pure prompt/JSON/answer helpers (no heavy deps)
  data.py                 # tokenizer/dataset loading + deterministic sampling
  model.py                # 4-bit model loading + LoRA attachment
  metrics.py              # ToolCallingMetrics + scorers (§14)
  train.py                # SFTTrainer wrapper + merge/publish
scripts/
  profile_tool_distribution.py # §8.1
  dataset_stats.py          # §8.2
  statistical_analysis.py    # §18
  capture_environment.py     # §12
  publish_dataset.py         # §28
  build_preprint.py         # PDF build
tests/                     # pytest: 29 tests (config invariants + formatting +
metrics)
```

Library	Purpose
Transformers / Datasets	Model/tokenizer; dataset loading
PEFT	LoRA / parameter-efficient fine-tuning
TRL	Supervised fine-tuning (SFTTrainer)
bitsandbytes	Quantization
PyTorch	Model execution
W&B	Experiment tracking
Hugging Face Hub	Artifact publication

This addresses the earlier observation that the pipeline should be modular rather than one end-to-end script: the stage functions now live in importable modules with unit tests, and `tests/test_config.py` pins the code's configuration to the paper's §11 table.

13.4 Code explanation quality

Each module is documented at two levels: a module docstring stating its role in the pipeline, and per-function docstrings stating pre/post-conditions. The two subtlest functions are explained inline:

- `formatting.extract_json` (§14/§21.7): three-layer parse — strip json `` fences, `tryjson.loads` on the whole string, then a balanced-brace scan of the first region; non-dict parses (e.g., a bare JSON list) are rejected because the contract is an `*object*`, preventing a downstream `pred.get("name")` crash (this exact bug was caught by `tests/test_formatting.py`).
- `metrics.evaluate_tool_calling(..., return_details=True)` (§18): returns per-example `{gt, raw, pred}` records so the paired McNemar/bootstrap test can be computed — a requirement that aggregate percentages alone cannot satisfy.

Run the suite with:

```
pip install pytest
python -m pytest tests/ -v      # 29 passed
```

14. Evaluation Framework and Metrics

Three metrics are computed over the 200-example validation split:

Metric	Definition	Computation
JSON validity	Output contains a parseable JSON object	JSON extracted via regex/substring match, then <code>json.loads()</code> ; not a raw-output purity check (§21.7)
Tool-name accuracy	Predicted name equals ground truth	Exact, case-sensitive string match
Argument exact match	Predicted arguments equals ground truth	Exact match on keys and values; no partial credit

Why exact match, not similarity scoring. In a real execution pipeline, a partially correct argument set (right tool, wrong value) still fails downstream. Exact match reflects the deployment failure mode more honestly than a softer metric, at the cost of not distinguishing "close" from "way off" failures.

Quantization control. By default the baseline is evaluated on the same 4-bit NF4 quantized base (`eval_load_in_4bit=True`) as the fine-tuned model, so the comparison isolates fine-tuning rather than precision. Scoring the bf16 base instead is a one-flag ablation (§25.4).

Not measured. Latency, token-level calibration, out-of-schema requests, and multi-tool selection when more than one tool could validly answer — candidates for a follow-up pass (§32).

15. Validation Strategy

The current strategy, and its known weaknesses:

1. **Fixed seed (42), deterministic split** — 5,200 sampled, 5,000/200 train/val.
Deterministic and reproducible (§28), but a single seed means variance across seeds is unmeasured (§21.3).
2. **Prompted-baseline control** — the base model is evaluated on the identical prompts and metrics as the fine-tuned model, so improvements are attributable to fine-tuning, not to prompt or metric changes.
3. **Paired comparison** — both models are scored on the same 200 examples, enabling McNemar's test and a paired bootstrap (§18); this is stronger than comparing two independent samples.
4. **Retention probe (GSM8K, n=50)** — a general-reasoning check under a fixed harness for both models.

The protocol to make results generalizable (V1 → V2):

#	Step	Artifact
V1	Lock an independent test split (e.g., a further 500 examples from the 60K, seed 7) before any further tuning; evaluate only after all decisions	held-out test set
V2	Re-run §8.1 and §8.2 profiling; report the three distribution quantities and truncation count	§8.1/§8.2 tables filled
V3	Run the full GSM8K (1,319 test examples) under the shared harness	retention table
V4	Run ≥ 3 seeds; report mean $\pm$ std	seed-variance table
V5	Produce the per-example dump and compute McNemar + bootstrap CI (§18)	<code>eval_predictions.jsonl</code>
V6	Record the environment (§12) and pin dependency versions	<code>environment.json</code> , lockfile

This section is the honest statement of what would make the results generalizable rather than descriptive.

16. Baseline Results

Metric	Result
JSON validity	78.5%
Tool-name accuracy	65.0%
Argument exact match	42.0%
GSM8K (n = 50)	52.0%

The base model often understands the request but adds markdown, explanatory text, selects the wrong tool, or omits arguments — establishing a meaningful baseline rather than evaluating the fine-tuned model in isolation.

17. Comparative Analysis

17.1 Base vs. fine-tuned

Metric	Base	TinyToolCaller	Improvement
JSON validity	78.5%	98.0%	+19.5 pp
Tool-name accuracy	65.0%	92.5%	+27.5 pp
Argument exact match	42.0%	84.0%	+42.0 pp

Dimension	Base model	TinyToolCaller
Argument exact match	42.0%	84.0%
Additional training cost	—	≈5,000 examples, single GPU, QLoRA (hours, not days)
Additional inference cost	—	Negligible (LoRA adapter adds minimal latency/memory)

Caveat — JSON validity. Computed after JSON extraction, not on the raw output. It measures whether a valid JSON object could be extracted, not whether the model produced pure JSON (§21.7).

Caveat — evaluation set. All figures come from the same 200-example split used during development; no independent held-out test set was held back. These are **in-sample experimental results, not unbiased estimates of generalization** (§15, §21.1).

Caveat — tool distribution (open item). The subset's tool-name distribution has not yet been profiled (§8.1). Until (a) unique-tool count, (b) top-10 frequency, and (c) train/validation match are reported, the tool-name-accuracy figure cannot be separated from possible skew toward high-frequency tools.

17.2 Position relative to related work

TinyToolCaller is **not directly comparable** to the models below (different data, scale, and benchmarks), but its position clarifies the contribution:

Work	Scale	Method	Data	Evaluation	What TinyToolCaller adds
xLAM-1b-fc-r [1][15]	1B	Full SFT	APIGen 60K	BFCL (AST)	Same data family, but prompted-baseline ablation at 1.5B with per-failure-mode metrics
Octopus v2 [14]	2B	Full SFT + functional tokens	Proprietary	Task accuracy + latency	QLoRA (not full FT); open recipe
NexusRaven-V2 [5]	13B	SFT (no GPT-4 distill)	Curation + retrieval	Nexus/API-Bank	Much smaller; isolates QLoRA-only lift
Granite-20B-FC [7]	20B	Multi-task granular SFT	Multiple	BFCL + 5 suites	Small-model focus; actionable metric decomposition
ToolACE [6]	8B	SFT on ToolACE	26.5K tools	BFCL/API-Bank	Confirms its "small raw models improve sharply with FT" at 1.5B
BFCL [2]	—	—	—	AST/execution	Independent benchmark is a planned next step (§32), not yet run

The largest observed improvement is **argument exact match** (42.0% → 84.0%), the most deployment-relevant of the three. The direction is consistent across all three task-specific metrics, but the exact percentage-point improvements should not be read as precise estimates of unseen-data performance.

18. Statistical Analysis

All three task-specific improvements are **large by any reasonable reading**, even after accounting for estimation uncertainty. Wilson 95% confidence intervals for each reported proportion ($n = 200$; GSM8K $n = 50$):

Metric	Base	95% CI (base)	Fine-tuned	95% CI (ft)	Cohen's h	Effect
JSON validity	78.5%	[72.3%, 83.6%]	98.0%	[95.0%, 99.2%]	+0.68	medium
Tool-name accuracy	65.0%	[58.2%, 71.3%]	92.5%	[88.0%, 95.4%]	+0.71	medium
Argument exact match	42.0%	[35.4%, 48.9%]	84.0%	[78.3%, 88.4%]	+0.91	large
GSM8K retention	52.0%	[38.5%, 65.2%]	50.0%	[36.6%, 63.4%]	−0.04	negligible

How to read this. The fine-tuned model's confidence intervals do not overlap the base model's for any of the three task metrics, so the direction of improvement is robust to sampling noise at the reported proportions. Cohen's h classifies the argument-exact-match gain as **large** and the other two as **medium**; the GSM8K change is negligible.

Paired significance testing. Because both models are evaluated on the same 200 examples, the correct test for the difference is **McNemar's test** on the discordant pairs (base-wrong/fine-tuned-right vs. base-right/fine-tuned-wrong), plus a **bootstrap confidence interval** on the paired difference. Neither can be computed from aggregate percentages — it requires the per-example outcomes. The pipeline writes these via `train_tool_caller.py --eval-dump`, and `scripts/statistical_analysis.py --mcnemar` computes McNemar, the bootstrap CI, and Cohen's h from them. Until that file is produced from a real run, the statement is: the CIs above support the direction and approximate magnitude of the effect; the exact paired p-value is pending the per-example dump.

```
python train_tool_caller.py --eval-dump outputs/eval_predictions.jsonl
python scripts/statistical_analysis.py --mcnemar outputs/eval_predictions.jsonl
```

GSM8K. The 2-point change on 50 examples is within sampling noise (95% CI half-width $\approx \pm 13$ pp at $p \approx 0.5$), so the experiment **cannot distinguish "no forgetting" from "moderate forgetting"** and must not be cited as evidence of retention either way.

19. Results Interpretation and Error Analysis

The three metrics decompose the failure surface into bands that call for different engineering responses.

19.1 Success stories

1. **Structured output.** JSON extraction validity 78.5% $\rightarrow$ 98.0%: the model became considerably more consistent at the structured-output objective on the evaluated examples. (This is after extraction — see §21.7 — so it is not "98% of raw responses are pure JSON".)
2. **Tool selection.** 65.0% $\rightarrow$ 92.5%: improved association of requests with the correct available function.
3. **Argument construction.** 42.0% $\rightarrow$ 84.0%: the largest gain, and the most practically important — selecting the right function is insufficient if the parameters are wrong.

19.2 Failure analysis

Interpreting the fine-tuned model's 200-example results as a failure budget:

Failure class	Estimated share	Production consequence	Mitigation
Invalid / non-extractable JSON	≈ 2% (4/200)	Call cannot even be parsed	Retry-with-repair; schema validation (§22)
Valid JSON, wrong tool	≈ 7.5% (15/200)	Wrong function executed	Tool allowlist; relevance detection
Right tool, wrong arguments	≈ 8.5 pp gap (185 correct tools vs. 168 correct arguments)	Silent semantic error — well-typed but wrong values	Type/range/semantic validation; canary testing

The third row is the most consequential insight: **schema validation cannot catch it**, because the arguments are structurally valid. The gap between tool accuracy (92.5%) and argument exact match (84.0%) is exactly the band where the model picks the right tool but fills it wrong — the failure class that dominates production risk and that a single aggregate accuracy number would hide. (Exact joint counts require the per-example dump; the figures above are derived from the aggregates under the assumption that argument correctness implies tool correctness, which holds in practice.)

19.3 Illustrative transcripts

The transcripts below are **reconstructions of the documented failure modes (§1)**, not logged model outputs; the per-example dump (§18) produces the real ones.

Success (fine-tuned):

```
User Request: What's the weather in Tokyo?
Available Tools: [get_weather(location: string, unit: celsius|fahrenheit)]

Model output: {"name": "get_weather", "arguments": {"location": "Tokyo", "unit": "celsius"}}
Scored: JSON valid ✓ · tool correct ✓ · arguments exact ✓
```

Failure A — markdown-wrapped (baseline-typical):

```
Model output:
```json
{"name": "get_weather", "arguments": {"location": "Tokyo"}}
```

Scored: JSON valid (after fence-stripping) ✓ · tool correct ✓ — but raw output is not pure JSON (§21.7)

```
Failure B — wrong tool:

```text
User Request: Where is order 12345?
Model output: {"name": "search_products", "arguments": {"query": "12345"}}
Ground truth: {"name": "get_order_status", "arguments": {"order_id": "12345"}}
Scored: JSON valid ✓ · tool ✗ · arguments ✗
```

Failure C — right tool, hallucinated argument (the dangerous one):

```
User Request: What's the weather in Tokyo?
Model output: {"name": "get_weather", "arguments": {"location": "Tokyo", "unit": "metric"}}
Ground truth: {"name": "get_weather", "arguments": {"location": "Tokyo", "unit": "celsius"}}
Scored: JSON valid ✓ · tool ✓ · arguments ✗ ← "metric" is not in the schema enum
```

20. Catastrophic Forgetting Analysis

Model	GSM8K (n = 50)
Base	52.0%
Fine-tuned	50.0%

A 2-point change on 50 examples is well within sampling noise (95% CI half-width $\approx \pm 13$ pp). This experiment **cannot distinguish "no forgetting" from "moderate forgetting"** and should not be cited as evidence of retention either way. A stronger analysis would use the full GSM8K (or another benchmark) under a fixed, identical harness for both models.

21. Limitations

Limitation	Likely impact on headline results
No independent test set (§21.1)	High — in-sample figures likely overstate generalization by an unknown margin
Small GSM8K sample (§21.2)	High — the retention claim is statistically unsupported either way
No hyperparameter sweep (§21.3)	Medium — gains may be improvable or a local optimum
Limited training data, 5K/60K (§21.4)	Medium — more data would likely improve tool coverage
Single-turn focus (§21.5)	Low for this report — out of scope, doesn't bias current numbers
No external benchmark (§21.6)	Medium — limits comparability (BFCL, τ -bench)
JSON-validity extraction leniency (§21.7)	High — 98.0% overstates raw-output compliance
Tool distribution unprofiled (§21.8)	High — until §8.1 is filled in, tool accuracy may be inflated by a skewed subset

21.1 The 200-example validation split is also the final evaluation set; a future version should maintain train / validation / independent test.

21.2 50 GSM8K examples; the 2-point change is not a precise measurement of degradation.

21.3 Fixed configuration; no systematic search over learning rate, LoRA rank, dropout, epochs, sequence length, or batch configuration.

21.4 5,000 examples of 60,000.

21.5 Multi-turn tool use is not addressed.

21.6 No results yet on a standardized external function-calling benchmark (BFCL [2], τ -bench [8]).

21.7 JSON is extracted before scoring, so the JSON-validity metric does not represent the stricter requirement that the raw output contain only JSON. Correct this in a future evaluation version.

21.8 The subset's tool-name distribution (unique count, top-10 concentration, train/validation match) has not yet been measured (§8.1). Until those three quantities are reported, the tool-name-accuracy figure is entangled with possible selection skew, and the validation split's representativeness is unverified. **Highest-priority measurement to complete before publication.**

22. Deployment Considerations

TinyToolCaller is suitable for experimentation and model-level inference; production tool execution requires additional infrastructure.

Control	Priority	Rationale (tied to measured failure rate)
JSON Schema validation	Critical	$\approx 2\%$ of fine-tuned outputs are not valid JSON — catch before execution
Tool allowlist	Critical	$\approx 7.5\%$ tool-selection error means wrong-but-valid calls occur; allowlisting limits blast radius
Argument validation (type/range)	Critical	$\approx 16\%$ argument mismatch — schema validation alone won't catch semantically wrong-but-well-typed values
Authorization / scoping	High	Independent of model accuracy — required regardless
Retry-with-repair	Medium	Could recover some of the $\approx 2\%$ JSON-invalid cases cheaply
Audit logging	Medium	Post-hoc failure analysis
Rate limits / timeouts	Standard	Generic API hygiene

A minimal validator for the model's output (the "Critical" row, in ~ 15 lines):

```
import jsonschema

def validate_and_execute(raw: str, tool_schema: dict, executor, user_id: str):
    call = extract_json(raw)                # §14 parsing
    if call is None:
        raise InvalidCall("unparseable")    # → retry-with-repair path
    if call["name"] not in TOOL_ALLOWLIST:   # allowlist, not the model
        raise ForbiddenTool(call["name"])
    jsonschema.validate(instance=call["arguments"],
                        schema=tool_schema["parameters"]) # type/range
    if not authorized(user_id, call["name"], call["arguments"]):
        raise Unauthorized(call)
    return executor[call["name"]](**call["arguments"])
```

A safe production architecture:

```
User → Application → Tool Registry + Prompt Builder → TinyToolCaller → JSON output
    → JSON Schema Validator → Authorization → Tool Executor → External API
```

The model should not be granted unrestricted execution privileges.

23. Monitoring and Maintenance

Starting alert thresholds (calibrated against this experiment's baseline; revise after real traffic):

Signal	Threshold	Action
JSON validity	< 95%	Page/alert (2 pp below the 98.0% eval figure)
Tool-selection failure rate	> 10%	Investigate for drift (observed: 7.5%)
Unknown/out-of-schema tool requests	> 1% of traffic	Users are asking for capabilities outside the trained tool set

Metric definitions (what to log per request): raw output, extraction result, predicted name, predicted arguments, ground-truth (when available), validity flag, latency_ms, and the tool-set size — the last one matters because §13.2 shows longer tool sets truncate and correlate with harder prompts.

Data-drift detection. Two complementary checks: (i) a categorical chi-square test comparing the production tool-name distribution against the training distribution (§8.1's method, reused at inference time); (ii) an embedding-distance or n-gram novelty score on production queries vs. the training set. Either diverging from baseline signals that the request mix has moved off-distribution.

Cadence. Weekly failure-log review; a monthly regression run of the full evaluation suite against any new model version before promotion. The current project tracks training loss, learning rate, baseline/fine-tuned metrics, and system metrics (GPU utilization/memory) in W&B. A production system extends this to **model metrics** (validity, tool-selection and argument-validation failures, unknown-tool requests, retry rate, generation latency), **infrastructure metrics** (GPU memory, CPU/GPU utilization, throughput, request latency, error rate), and **data drift**.

Maintenance loop:

```
Production requests → failure analysis → evaluation dataset → regression test
→ fine-tuning → new model version → canary evaluation → deployment
```

Runbook outline. (1) Triage: classify failures by §19.2's table; (2) if JSON-validity drops, check prompt/tokenizer changes first (cheapest); (3) if tool-selection drifts, run the §8.1 chi-square and check for new tool categories; (4) if argument errors rise, sample 20 failures and check for a common hallucinated parameter; (5) promote a fix only after the monthly regression passes and a canary shows no regression.

24. Significance and Implications of the Work

The significance is **not** that a 1.5B model becomes universally more capable. The defensible finding, with three concrete implications:

A small open-weight model can be substantially specialized for a narrowly defined structured-output task using parameter-efficient fine-tuning.

1. **Engineering implication.** Task specialization can be more valuable than increasing model size when the target is narrow and measurable. The +42 pp argument-exact-match gain is the headline, but the practical claim is cost-shaped: a 1.5B QLoRA adapter trains in hours on one GPU and adds negligible inference latency over the base (§17.1), versus a much larger model whose marginal capability may be unneeded for a fixed tool registry.
2. **Systems implication.** The LLM should be one component of a tool-calling system, not the whole agent. The failure budget (§19.2) shows that even at 98% validity, production safety comes from the deterministic layer — allowlist, schema validation, authorization — not from the model alone. This separation makes the system testable and auditable in ways a monolithic agent is not.
3. **Scientific implication (small).** The result corroborates, at 1.5B, ToolACE's observation that small raw models have minimal function-calling ability but improve sharply with fine-tuning [6] — and does so with a prompted-baseline control and confidence intervals, which leaderboard-centric reports often omit.

25. Industry Insights

The shifting problem definition. As agentic systems interact with APIs, databases, search, calendars, and enterprise applications, the practical question moves from "can the model generate a good answer?" to "can the model reliably produce an action software can safely execute?" — and the failure-cost curve changes with it: a verbose-but-correct answer costs nothing, while a wrong tool call can trigger an irreversible external effect. TinyToolCaller addresses one component of that transition.

The large-generalist vs. small-specialist trade-off. A small specialized model is attractive when the operational task is narrow enough that general reasoning is not the primary requirement — the situation for a fixed enterprise tool registry. The decision hinges on task and schema complexity, latency, cost, error tolerance, deployment environment, and safety requirements; this project does **not** establish that smaller models are universally better.

The compression and on-device trend. Octopus v2 [14] and the small-model function-calling study [10] reflect a broader push toward on-device and edge tool use, where a 1.5B QLoRA adapter is a natural fit; the project's single-GPU, hours-not-days training story (§17.1) is aimed exactly at that segment.

The open-weight function-calling ecosystem. xLAM-1b/7b-fc-r [1], Gorilla [3], NexusRaven [5], Hermes, Granite [7], and Qwen's own function-calling variants show open models converging on proprietary tool-calling parity; the remaining differentiators are data quality (APIGen's execution verification [1]) and evaluation rigor (BFCL's AST scoring [2]) — which is why §32 prioritizes joining that external evaluation.

26. Uncommon Insights

Observations from this work that are not obvious from the headline table:

1. **The cheapest metric was the least valuable, and the most valuable metric improved the most.** JSON validity (nearly solved at 98%) is the metric with the smallest practical consequence, because malformed JSON is cheaply recoverable (retry/repair). Argument exact match — the failure class that silently breaks production — improved the most (+42 pp). The model didn't just learn formatting; it learned schema-following.
2. **The 92.5% → 84.0% gap is the real deployment risk, and it is invisible to schema validation.** The ≈ 8.5 pp of examples with the right tool but wrong arguments produce well-typed, structurally valid output. Only semantic or range validation can catch them — a cost that most function-calling benchmarks, which score "call correctness" in aggregate, do not surface.
3. **In-sample baselines can be misleading in both directions.** The baseline's 78.5% "JSON validity" already includes fence-stripping cleanup; conversely, its 42% argument rate shows most of the gap was never about formatting. Report the raw-output metric separately to see both effects (§21.7).
4. **A quantized baseline is a control, not a convenience.** Evaluating the base model in the same 4-bit NF4 regime removes precision as a confound; if the "improvement" partially disappears when the baseline is bf16, that itself is a finding about quantization, not fine-tuning. This ablation is one flag away (`eval_load_in_4bit=False`).
5. **Truncation is a hidden schema-size confound — and it's measurable.** With the real tokenizer, ~ 5 verbose tools already approach the 1024-token cap and 10 exceed it (§13.2). If the eval set skews the same way, accuracy is overstated relative to production prompts with many tools. `scripts/dataset_stats.py` turns this from speculation into a number.
6. **Retention claims need more than a 50-example diff.** A ± 13 pp confidence interval means even a true 10-point forgetting effect would be undetectable here — the retention "result" is currently an absence of evidence, not evidence of absence.
7. **Tests caught a real bug cheaply.** The suite's `test_extract_json_rejects_bare_list` exposed that a JSON-list output would crash the scorer (`.get` on a list); the fix (reject non-dict parses) is two lines. A 29-test suite is disproportionate value for a project this size, and it runs on a CPU with no GPU stack because the heavy imports are lazy (§13.2).

27. Source Credibility and Provenance

The project relies on first-party sources: the xLAM dataset is published by Salesforce AI Research with documented APIGen generation and verification [1]; Qwen2.5-1.5B-Instruct is published by Qwen (Apache-2.0); QLoRA and LoRA follow the original peer-reviewed work [11][12]; TRL/PEFT documentation comes from Hugging Face [16]; GSM8K originates from Cobbe et al. [13]; the benchmark and model comparisons cite BFCL [2], ToolACE [6], Gorilla [3], Toolformer [4], NexusRaven [5], Granite [7], τ -bench [8], and Octopus v2 [14].

28. Licensing and Attribution

Artifact	License
Source dataset (Salesforce/xlam-function-calling-60k)	CC-BY-4.0
Base model (Qwen/Qwen2.5-1.5B-Instruct)	Apache-2.0
TinyToolCaller (code, derived data, documentation, artifacts)	Project-specific (see LICENSE)

Users should review upstream licenses and attribution requirements before redistribution or commercial deployment.

29. Reproducibility and Verifiability

Falsifiable claims. The paper's central claims are stated so that a third party can attempt to refute them: (i) on the documented 5,200-example seed-42 subset, QLoRA raises each of the three metrics by the reported magnitudes (§17); (ii) the direction of improvement holds under paired testing (§18); (iii) the GSM8K change is indistinguishable from noise (§20).

Determinism and configuration.

Seed 42 · 5,000 train / 200 validation · LoRA $r=16$, $\alpha=32$, dropout=0.05
LR $2e-4$ · cosine + 3% warmup · effective batch 16 · 2 epochs · max seq len 1024

Workflow.

```

pip install -r requirements.txt
pip install pytest

export HF_TOKEN=<your_huggingface_token>      # gated access to the source dataset
export WANDB_API_KEY=<your_wandb_key>          # optional

python -m pytest tests/ -v                    # 29 tests (config, formatting, metrics)
python scripts/capture_environment.py --save outputs/environment.json
python scripts/dataset_stats.py                # §8.2
python scripts/profile_tool_distribution.py     # §8.1
python train_tool_caller.py                   # full 14-stage pipeline
python train_tool_caller.py --eval-dump outputs/eval_predictions.jsonl
python scripts/statistical_analysis.py --report
python scripts/statistical_analysis.py --mcnemar outputs/eval_predictions.jsonl
python scripts/publish_dataset.py --push       # publishes the derived dataset

```

Verifiability gaps to close before final publication. (1) Record the full environment (§12), pin exact dependency versions (a lockfile), and publish a container/requirements hash; (2) publish checksums for the derived `train.parquet` / `validation.parquet`; (3) commit the W&B run ID/loss curve for the reported training run; (4) fix the `datasets` version used for the seed-42 shuffle (the shuffle RNG is version-sensitive, so §8.1's numbers are only exactly reproducible on the recorded version).

30. Repository and Dataset

Artifact	Location
Code	https://github.com/strdst7/TinyToolCaller
Project dataset	https://huggingface.co/datasets/strdst77/TinyToolCaller
Source dataset	https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k
Base model	https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct

31. Project Architecture

```

Salesforce xLAM 60K → sampling/split (5K train / 200 val) → ChatML formatting
  → Qwen2.5-1.5B-Instruct (4-bit NF4 + QLoRA) → SFTTrainer/TRL
  → LoRA adapter weights
    ↳ function-calling evaluation (3 metrics + per-example dump)
    ↳ GSM8K retention
    ↳ statistical analysis (Wilson CI / McNemar / bootstrap)
  → model comparison → adapter + base → merged model → Hugging Face Hub

```

Technical progression. The project moved from a single end-to-end script to a modular package (`tinytoolcaller/`) with unit tests that pin the configuration to the paper (§13.3); from a single aggregate "accuracy" to three failure-mode metrics (§14); and from point estimates to confidence intervals and a paired significance test (§18). The next step in that progression is external, independent evaluation (§32).

32. Future Architecture

A production-ready evolution adds a deterministic runtime around the model:

```
User → Application → Tool Registry → Prompt Builder → TinyToolCaller → generated JSON
    → JSON Schema Validator → (invalid → repair/reject) → Authorization
    → Tool Executor → External API → tool result
```

The current project produces the **tool-call generation component**; a production agent adds validation, execution, permissions, retries, and observability.

33. Future Directions and Research Extensions

Ordered by effort vs. impact:

Priority	Direction	Effort	Impact
1	Independent held-out test split (§15-V1)	Low	High — converts in-sample to generalization evidence
2	Tool-distribution profiling (§8.1) + basic stats (§8.2)	Low	High — removes the skew caveat
3	Full GSM8K retention run (§15-V3)	Low	High — makes the retention claim meaningful
4	Raw-output (no-extraction) JSON metric (§21.7)	Low	Medium — deployment-relevant purity number
5	Quantization ablation: bf16 vs. 4-bit baseline (§25.4)	Low	Medium — isolates quantization vs. fine-tuning
6	Multi-seed variance (§15-V4)	Low	Medium — honest error bars
7	Hyperparameter sweep (rank, LR, epochs)	Medium	Medium — may improve gains
8	External benchmark (BFCL [2], τ -bench [8])	Medium	High — comparability
9	Multi-turn tool use (APIGen-MT [9] direction)	High	High — production agentic relevance
10	Distillation from a larger teacher	High	High — capability transfer

Larger research bets: multi-turn tool use, distillation, runtime schema validation, 8-bit/4-bit inference optimization, and relevance detection ("when not to call") à la BFCL V3 [2].

34. Accessibility and Learning Design

The project is structured for readers with basic Python and ML knowledge but no prior LLM fine-tuning experience. The workflow is intentionally simple — data → format → baseline → fine-tune → evaluate → compare → publish — and readers need not

understand every Transformer implementation detail. The concepts that matter: what function calling is; why structured JSON matters; what supervised fine-tuning does; what LoRA does; why quantization reduces memory; how baseline and post-training evaluation differ; why confidence intervals and validation design determine the strength of conclusions.

34.1 Glossary

Term	Meaning in this project
Function calling / tool calling	Generating a structured request (name + arguments) that a program can execute
Fine-tuning (SFT)	Training on input→target examples to shape behaviour
LoRA	Freezing the base weights and training small low-rank update matrices
QLoRA	LoRA on top of a 4-bit-quantized (NF4) base model to cut memory
Adapter	The trained LoRA weights, merged or loaded alongside the base
ChatML	The <code>< im_start >/< im_end ></code> message format used by Qwen
Baseline	The unmodified base model scored on the same prompts
JSON validity (this project)	Whether a JSON object could be extracted from the output (§14)
Exact match	Predicted name / arguments equal ground truth exactly (§14)
Wilson CI	A confidence interval for a proportion (here, 95%)
McNemar's test	Paired significance test for before/after on the same examples
GSM8K	Grade-school math benchmark used as a retention probe
Gated dataset	Requires accepting terms + a Hugging Face token to download

35. Key Takeaways

1. **Small models can be specialized** — a 1.5B model can become substantially better at a narrow structured-output task.
2. **QLoRA makes specialization accessible** — parameter-efficient fine-tuning reduces the trained state.
3. **Function calling is more than valid JSON** — tool selection and argument correctness are separate failure dimensions.
4. **Evaluation design matters** — evidence is only as strong as the validation methodology, and statistics should carry confidence intervals.
5. **LLMs should not be the entire agent** — combine the LLM with validation, authorization, and deterministic execution.

36. Conclusion

Starting from Qwen/Qwen2.5-1.5B-Instruct and fine-tuning with QLoRA on a 5,000-example subset of the Salesforce xLAM dataset, TinyToolCaller reports **JSON validity 78.5% →**

98.0%, tool accuracy 65.0% → 92.5%, argument exact match 42.0% → 84.0% on its 200-example evaluation split, with a 50-example GSM8K retention check moving 52.0% → 50.0%. The contribution is not a claim of universal superiority but a demonstrable engineering pattern:

General-purpose model → task-specific data → parameter-efficient fine-tuning
→ specialized small model → structured interface → deterministic software

This pattern suits lower-cost, lower-latency AI systems whose target capability is narrow, measurable, and operationally well-defined. TinyToolCaller is best viewed as a **reproducible applied LLM engineering study and a foundation for a production-grade tool-calling runtime** — not a finished autonomous-agent platform.

37. Publication Checklist

- [x] Clear problem statement (§1)
- [x] Related work with critical analysis and current citations (§2)
- [x] Objectives, contributions, and originality stated (§3)
- [x] Assumptions and scope stated (§4)
- [x] Intended audience and use case (§5)
- [x] Dataset source, selection rationale, and description (§8)
- [x] Worked ChatML example with real tokenizer counts (§9.1)
- [x] Dataset processing methodology (§9)
- [x] Training methodology, parameters, and per-parameter rationale (§10–§11)
- [x] Experimental environment template + capture script (§12)
- [x] Implementation workflow, package layout, and code quality (§13)
- [x] Unit tests passing (29) (§13.3–§13.4)
- [x] Evaluation framework and metrics (§14)
- [x] Validation strategy and protocol (§15)
- [x] Baseline established (§16)
- [x] Comparative results + related-work position (§17)
- [x] Statistical analysis with confidence intervals (§18)
- [x] Results interpretation, error analysis, illustrative transcripts (§19)
- [x] Limitations disclosed (§21)
- [x] Deployment considerations with validator snippet (§22)
- [x] Monitoring, drift detection, and runbook (§23)
- [x] Significance, industry insights, uncommon insights (§24–§26)
- [x] Source credibility and licensing (§27–§28)
- [x] Reproducibility and verifiability instructions (§29)
- [x] Code, dataset, and model linked (§30)
- [x] Future architecture and research roadmap (§32–§33)
- [x] Accessibility and glossary (§34)

- [x] Rubric coverage matrix (§38)
- [] **Tool-distribution profiling (§8.1) completed** (open — blocks unqualified quoting of tool accuracy)
- [] Basic dataset statistics computed and recorded (§8.2) (open)
- [] Independent test set created and locked before model selection (open)
- [] Full-size GSM8K retention run (open)
- [] Experimental environment recorded; dependency versions pinned (§12, §29) (open)
- [] Paired McNemar + bootstrap computed from the per-example dump (§18) (open)

38. Rubric Coverage Matrix

Where each evaluation dimension is addressed.

Applied solution showcase

Rubric item	Section(s)
Evaluation Framework	§14, §15
Dataset Description	§8, §8.2, §9.1
Dataset processing Methodology	§9, §9.1
Implementation Considerations	§13.2
Deployment Considerations	§22
Monitoring and Maintenance Considerations	§23
Comparative Analysis	§17
Results Interpretation	§19
Future Directions	§33
Purpose-Aligned Topic Coverage	§1, §3, §38
Code Usage Appropriateness	§13.3
Code Clarity and Presentation	§13.3
Code Explanation Quality	§13.4
Industry Insights	§25
Success/Failure Stories	§19.1–§19.3
Technical Progression	§31
Source Credibility	§27
Uncommon Insights	§26
Significance and Implications of Work	§24

Research paper

Rubric item	Section(s)
Testability/Verifiability	§29
Literature Review Coverage & Currency	§2 (2021–2025, incl. BFCL ICML 2025, APIGen-MT 2025)
Literature Review Critical Analysis	§2.1–§2.4 ("Critical gap" notes)
Citation Relevance	§2, §27, §39 (numbered, claim-anchored)
Assumptions Stated	§4
Evaluation Framework	§14
Validation Strategy	§15
Dataset Description	§8, §9.1
Dataset Selection or Creation	§8.0
Dataset processing Methodology	§9
Basic Dataset Stats	§8.2
Implementation Details	§13
Parameters & Configuration	§11 (with rationale)
Experimental Environment	§12
Implementation Considerations	§13.2
Comparative Analysis	§17
Statistical Analysis	§18
Results Interpretation	§19
Future Directions	§33
Originality of Work	§3
Innovation in Methods/Approaches	§3
Code Usage Appropriateness	§13.3
Code Clarity and Presentation	§13.3
Content Accessibility	§34, §34.1
Significance and Implications of Work	§24

39. References

1. Liu, Z., Hoang, T., Zhang, J., et al. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets. arXiv:2406.18518, 2024.
2. Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., & Gonzalez, J. E. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. ICML 2025, PMLR 267:48371–48392.
3. Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. Gorilla: Large Language Model Connected with Massive APIs. arXiv:2305.15334, 2023.

4. Schick, T., Dwivedi-Yu, J., Dessì, R., et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023.
5. Srinivasan, V. K., Dong, Z., Zhu, B., et al. NexusRaven: A Commercially-Permissive Language Model for Function Calling. NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following, 2023.
6. Liu, Z., et al. ToolACE: Winning the Points of LLM Function Calling. arXiv:2409.00920, 2024 (ICLR 2025).
7. Abdelaziz, I., et al. Granite-Function Calling Model: Introducing Function Calling Abilities via Multi-task Learning of Granular Tasks. arXiv:2407.00121, 2024.
8. Sierra, S., et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045, 2024.
9. Prabhakar, A., Liu, Z., Zhu, M., et al. APIGen-MT: Agentic Pipeline for Multi-Turn Data Generation via Simulated Agent-Human Interplay. arXiv:2504.03601, 2025.
10. Small Models, Big Tasks: An Exploratory Empirical Study on Small Language Models for Function Calling. arXiv:2504.19277, 2025.
11. Hu, E. J., Shen, Y., Wallis, P., et al. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685, 2021.
12. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. arXiv:2305.14314.
13. Cobbe, K., Kosaraju, V., Bavarian, M., et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168, 2021.
14. Chen, W., & Li, Z. Octopus v2: On-device Language Model for Super Agent. arXiv:2404.01744, 2024.
15. Zhang, J., Lan, T., Zhu, M., et al. xLAM: A Family of Large Action Models to Empower AI Agent Systems. arXiv:2409.03215, 2024.
16. Hugging Face. TRL / SFTTrainer Documentation. https://huggingface.co/docs/trl/sft_trainer
17. Qin, Y., et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789, 2023.

40. Project Links and Citation

Code — <https://github.com/strdst7/TinyToolCaller> **Project dataset** — <https://huggingface.co/datasets/strdst77/TinyToolCaller> **Source dataset** — <https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k> **Base model** — <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>

```
@misc{tinytoolcaller,  
  title      = {TinyToolCaller: QLoRA Fine-Tuning of a 1.5B LLM for Reliable Function  
Calling},  
  author     = {strdst7},  
  year       = {2026},  
  howpublished = {\url{https://github.com/strdst7/TinyToolCaller}},  
  note       = {Base model: Qwen/Qwen2.5-1.5B-Instruct. Source dataset:  
Salesforce/xlam-function-calling-60k (APIGen).}  
}
```

Final Project Statement

TinyToolCaller demonstrates that a small open-weight language model can be deliberately specialized for reliable function calling through QLoRA, achieving substantial improvements in structured output, tool selection, and argument accuracy while maintaining a reproducible and transparent evaluation pipeline. The project provides a practical foundation for exploring low-cost LLM agents in which the model generates structured intent and deterministic software remains responsible for validation, authorization, and execution.

Nur Amirah Mohd Kamil | 2026 | Ready Tensor for LLM Fine-Tuning Specialist
Fine-tune and optimize an LLM using PEFT techniques.